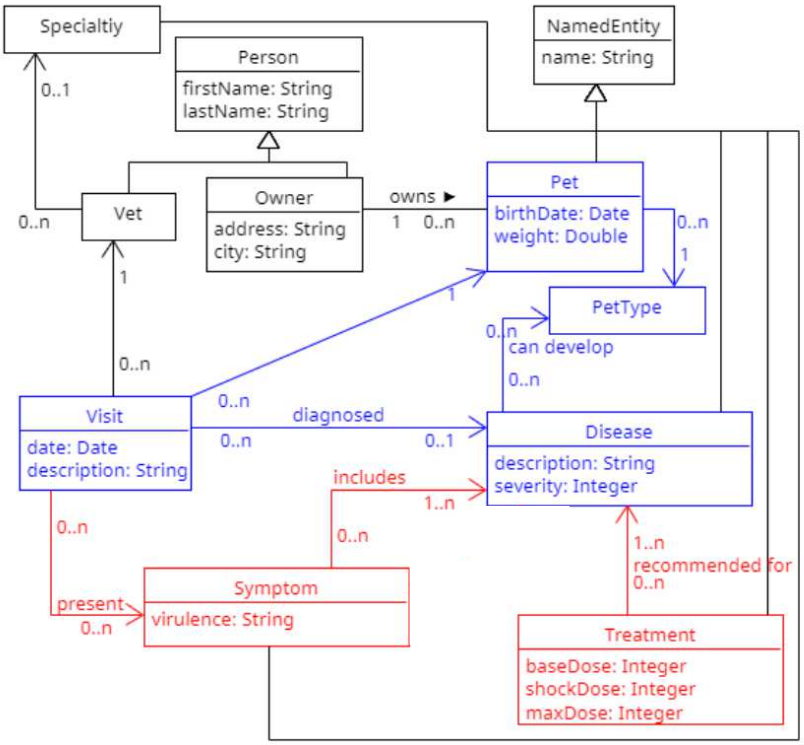
Control práctico de DP1 2024-2025 (Control-check 1)

Enunciado

En este ejercicio, añadiremos la funcionalidad de gestión de enfermedades, síntomas y tratamientos médicos. Concretamente, se proporciona una clase “Disease” que representa a las enfermedades que pueden desarrollar las mascotas, que se relaciona con el tipo de mascotas que pueden sufrirlas. Además, tendremos las clases “Symptom” y “Treatment” que representan a los síntomas que pueden aparecer a consecuencia de una enfermedad y los tratamientos recomendados para cada enfermedad respectivamente. Además, se ha creado una relación que indica qué síntomas son susceptibles de presentarse para una enfermedad llamada “includes”, para ayudar a los veterinarios a realizar diagnósticos más precisos.

El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:



Las clases para las que realizaremos el mapeo objeto-relacional como entidades JPA se han señalado en rojo. Las clases en azul son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control de laboratorio.

Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias. Si desea ver el resultado que arrojarían las pruebas en backend, puede ejecutarlas (bien mediante su entorno de desarrollo favorito, bien mediante el comando “*mvnw test*” en la carpeta raíz del proyecto). Cada ejercicio correctamente resuelto valdrá dos puntos, el número de casos de prueba de cada ejercicio puede variar entre uno y otro y la nota se calculará en base al porcentaje de casos de prueba que pasan. Por ejemplo, si pasan la mitad (50%) de los casos de prueba de

un ejercicio, usted obtendrá un punto (2*0,5=1), y si pasan un 10% obtendrá 0,2 puntos.Para comenzar esta prueba debe aceptar la tarea disponible en:

https://classroom.github.com/a/wuz35o_p

Al aceptar dicha tarea, se creará un repositorio único individual para usted, debe usar dicho repositorio para realizar el control práctico. Debe entregar la actividad en EV asociada al control check proporcionando como texto la dirección url de su repositorio personal. Recuerde que además debe entregar su solución del control.

La entrega de su solución al control se realizará mediante un único comando "*git push*" a su repositorio individual. Recuerde que debe hacer push antes de cerrar sesión en la computadora y abandonar el aula, de lo contrario, su intento se evaluará como no presentado. Su primera tarea en este control será clonar (recuerde que si va a usar los equipos del aula para realizar el control necesitará usar un token de autenticación de GitHub como clave, tiene un documento de ayuda a la configuración en el propio repositorio del control). A continuación, deberá importar el proyecto en su entorno de desarrollo favorito y comenzar los ejercicios abajo listados. Al importar el proyecto, el mismo puede presentar errores de compilación. No se preocupe, si existen, dichos errores irán despareciendo conforme usted vaya implementando los distintos ejercicios del control.

Nota importante 1: No modifique los nombres de las clases ni la signatura (nombre, tipo de respuesta y parámetros) de los métodos proporcionados como material de base para el control. Las pruebas que se usan para la evaluación dependen de que las clases y los métodos tengan la estructura y nombres proporcionados. Si los modifica probablemente no pueda hacer que pasen las pruebas.

Nota importante 2: No modifique las pruebas unitarias proporcionadas como parte del proyecto bajo ningún concepto. Aunque modifique las pruebas en su copia local del proyecto, éstas serán restituidas mediante un comando git previamente a la ejecución de las pruebas para la emisión de la nota final, por lo que sus modificaciones en las pruebas no serán tenidas en cuenta en ningún momento.

Nota importante 3: Mientras haya ejercicios no resueltos habrá tests que no funcionen y, por tanto, el comando "*mvnw install*" finalizará con error. Esto es normal debido a la forma en la que está planteado el control y no hay que preocuparse por ello. Si se quiere probar la aplicación se puede ejecutar de la forma habitual pese a que "mvnw install" finalice con error.

Nota importante 4: La descarga del material de la prueba usando git, y la entrega de su solución con git a través del repositorio GitHub creado a tal efecto forman parte de las competencias evaluadas durante el examen, por lo que no se aceptarán entregas que no hagan uso de este medio, y no se podrá solicitar ayuda a los profesores para realizar estas tareas.

Nota importante 5: No se aceptarán como soluciones válidas proyectos cuyo código fuente no compile correctamente o que provoquen fallos al arrancar la aplicación en la inicialización del contexto de Spring. Las soluciones cuyo código fuente no compile o incapaces de arrancar el contexto de Spring serán evaluadas con una nota de 0.

Nota importante 6: Excepto el ejercicio 5 (que dependen del 4) lo ejercicios del examen son independientes y puedes ser resueltos en cualquier orden.

Test 1 – Modificar el servicio de gestión de Visitas para que no permita guardar diagnósticos imposibles

Modificar el método save del servicio de gestión de visitas (VisitService) de manera que se lance la excepción (UnfeasibleDiagnoseExcepction) en caso de que se intente guardar una visita asociada a una enfermedad que no pueden desarrollar las mascotas del tipo de la que ha venido a la visita. Por ejemplo, imaginen una visita donde el dueño trae a su *serpiente* y el veterinario lo diagnostica con *juanetes en las patas traseras* (que no puede ser desarrollada por ese tipo de mascotas, las serpientes no tienen patas). El método debe ser transaccional y hacer rollback de la transacción en caso de que se lance dicha excepción. Nótese, que existen relaciones entre la clase Disease y la clase PetType que permiten indicar las enfermedades que pueden o no desarrollar las mascotas dependiendo de su tipo/especie.

Test 2 – Creación de servicios de gestión de los síntomas y los tratamientos y del controlador para devolver las enfermedades disponibles

Parte 2A: Servicios de Gestión (1 punto):

Modificar las clases "SymptomService" y "TreatmentService", para que sean un servicio Spring de lógica de negocio y proporcionen una implementación a los métodos que permitan, usando una implementación del repositorio correspondiente (SymptomRepository / TreatmentRepository) que será inyectada mediante Spring (usando la anotación "*@Autowired(required=false)*"):

1. Obtener todos los síntomas/tratamientos (*Symptom / Treatment*) usando método *getAll* en cada uno de los servicios correspondientes.
2. Grabar un síntoma/tratamiento (*Symptom / Treatment*) en la base de datos (método *save*).

Todos estos métodos **deben ser transaccionales**, pero las anotaciones asociadas deben realizarse a nivel de método, no a nivel de clase**.** No modifique por ahora la implementación del resto de métodos de los servicios, ni tampoco los repositorios (hasta que no implemente las entidades en el ejercicio 4 no podrá hacerlo correctamente).

Parte 2B: Controlador para API de Enfermedades (1 punto):

Crear un método en el controlador "*DiseaseController*" (alojada en el paquete org.springframework.samples.petclinic.disease) que permita devolver todas las enfermedades existentes. El método debe responder a peticiones tipo GET en la URL:

http://localhost:8080/api/v1/diseases

Así mismo debe crear un método para devolver una enfermedad concreta, este método debe responder a peticiones en la url http://localhost:8080/api/v1/diseases/X donde X es la Id de la enfermedad a obtener, y devolverá los datos de la enfermedad correspondiente. En caso de que no exista una enfermedad con el id especificado el sistema debe devolver el código de estado 404 (NOT_FOUND).

Estos endpoint de la API asociados a la gestión de enfermedades deberían estar accesibles únicamente para usuarios de tipo Vet (que tengan la authority "VET").

Test 3 – Creación de un componente React de listado de enfermedades

Modificar el componente React proporcionado en el fichero "frontend/src/disease/index.js" para que muestre un listado de las enfermedades disponibles en el sistema. Para ello debe hacer uso de la API lanzando una petición tipo GET contra la URL api/v1/diseases.

Este componente debe mostrar las enfermedades en una tabla (se recomienda usar el componente Table de reactstrap) incluyendo columnas para su nombre, y el conjunto de tipos de mascotas que pueden desarrollar la enfermedad. Para esto último, el componente debe mostrar en la celda asociada una lista (preferiblemente no ordenada <ul>) con el nombre de los tipos de mascotas susceptibles de desarrollar la enfermedad en la celda correspondiente.

Para poder lanzar este test y comprobar su resultado puede colocarse en la carpeta de frontend y ejecutar el comando npm test y pulsar 'a' en el menú de comandos de jest. Nótese que previamente debe haber lanzado al menos una vez el comando npm install para que todas las librerías de node estén instaladas.

Test 4 – Creación de las entidades Symptom y Treatment y sus repositorios asociados

Modificar las clases "Symptom" y "Treatment" para que sean entidades. Estas clases están alojadas en el paquete "org.springframework.samples.petclinic.disease", y deben tener los siguientes atributos y restricciones:

Para ambas clases:

- El atributo de tipo entero (Integer) llamado "id" actuará como clave primaria en la tabla de la base de datos relacional asociada a la entidad.
- Un atributo de tipo cadena de caracteres (String) llamado "name" obligatorio (no puede ser nulo), que debe tener una longitud mínima de 3 caracteres y máxima de 50 y que no puede estar formada por caracteres vacíos (espacios, tabuladores, etc.).

Para la clase Treatment:

- El atributo de tipo entero (Integer) llamado "*baseDose*", que representa el número de miligramos de tratamiento por kilogramo de peso del animal. Este atributo será *obligatorio* y tendrá un *valor mínimo de 1*.
- El atributo de tipo entero llamado (Integer) llamado "*shockDose*", que representa una cantidad fija de miligramos. En la lógica de negocio del sistema esta cantidad se añadirá al tratamiento en caso de que la enfermedad a tratar sea mortal (valor 5 para el atributo severity en la entidad Disease). El atributo *es opcional*, pero si toma valor, tendrá un *valor mínimo de 1*.
- El atributo de tipo entero (Integer) llamado "*maxDose* " que representa la dosis máxima de tratamiento que puede llegar a administrarse (en miligramos de tratamiento por kilogramo de peso del animal). Este *atributo es obligatorio* y tendrá un *valor mínimo de 1*.

Para la clase Symptom:

- El atributo de tipo cadena caracteres (String) llamado "virulence" opcional que únicamente podrá tomar tres valores "LOW","MEDIUM","HIGH"[1].

Modificar las interfaces "SymptomRepository" y "TreatmentRepository" alojadas en el mismo paquete para que extiendan a CrudRepository. No olvide especificar sus parámetros de tipo.

Elimine las anotaciones @Transient de los métodos y atributos que las tengan en las entidades creadas anteriormente, así como la del atributo symptoms de la clase Visit. Se pide crear las siguientes relaciones

[1] La expresión regular para implementar dicha restricción es "^(LOW)|(MEDIUM)|(HIGH)$"

entre las entidades. Cree una relación unidireccional desde “Visit” hacia “Symptom” que exprese la que aparece en el diagrama UML (mostrado en la primera página de este enunciado) respetando sus cardinalidades, usando el atributo “symptoms” de la clase “Visit”.

Además, se pide crear una relación unidireccional desde “Symptom” hacia “Disease” que represente laque aparece en el diagrama UML, tenga en cuenta la cardinalidad que tiene, usando el atributo “includes” en la clase “Symptom”. Debe asegurarse de que las relaciones expresan adecuadamente la cardinalidad que muestra el diagrama UML, por ejemplo, algunos atributos pueden ser nulos puesto que la cardinalidad es 0..n pero otros no, porque su cardinalidad en el extremo navegable de la relación es 1..n.

Finalmente, se pide crear una relación unidireccional desde “Treatment“ hacia “Disease” que represente la que aparece en el diagrama, usando como nombre de atributo recommendedFor. Debe asegurarse de que las relaciones expresan adecuadamente la cardinalidad que muestra el diagrama UML, por ejemplo, el atributo no puede ser nulo y es obligatorio, puesto que la cardinalidad es 1..n en el extremo de Disease.

Test 5 – Modificación del script de inicialización de la base de datos para incluir dos síntomas y dos tratamientos y sus relaciones

Modificar el script de inicialización de la base de datos, para que se creen los siguientes síntomas (Symptom) y tratamientos (Treatment):

Symptom 1:

- id: 1
- name: “Cough”
- virulence: “LOW”

Symptom 2:

- id: 2
- name: “Hair loss”
- virulence: “MEDIUM”

Treatment 1:

- id: 1
- name: “aspirin”
- baseDose: 12
- shockDose:null
- maxDose: 50

Treatment 2:

- id: 2
- name: “paracetamol”
- baseDose: 20
- shockDose: 40
- maxDose: 150

Además, debe modificar este script de inicialización de la base de datos para que:

- El *Treatment* cuyo id es 1 se asocie con la *Disease* con id=2
- El Treatment cuyo id es 2 se asocie con la Disease con id=1
- El Symptom cuyo id es 1 tenga como posibles enfermedades incluidas la Disease con ids 2 y 3.
- El Symptom cuyo id es 2 tenga como posibles enfermedades incluidas la Diseases con ids 1 y 3.
- La Visit cuyo id es 1 tenga asociados los síntomas 1 y 2.

Tenga en cuenta que el orden en que aparecen los INSERT en el script de inicialización de la base de datos es relevante al definir las asociaciones.

CONTROL CHECK 1 - GRUPO 1 - 24/25

TEST 1

```
@Transactional
public Visit save(Visit visit) throws
DataAccessException,UnfeasibleDiagnoseException {
   if (!visit.getDiagnose().getSusceptiblePetTypes()
         .contains(visit.getPet().getType()))
   {
         throw new UnfeasibleDiagnoseException();
   }
   return visitRepository.save(visit);
}
```

TEST 2 A

```
@Service
public class SymptomService {
   SymptomRepository repo;

   @Autowired
   public SymptomService(SymptomRepository sr){
       this.repo=sr;
   }

   @Transactional(readOnly = true)
   public List<Symptom> getAll() {
       return repo.findAll();
   }

   @Transactional
   public Symptom save(Symptom s) {
       return repo.save(s);
   }
}
```

```
@Service
public class TreatmentService {
   private TreatmentRepository repo;

   @Autowired
   public TreatmentService(TreatmentRepository tr){
       this.repo=tr;
   }
```

```
    @Transactional(readOnly = true)
    public List<Treatment> getAll() {
        return repo.findAll();
    }

    @Transactional
    public Treatment save(Treatment t) {
        return repo.save(t);
    }
}
```

TEST 2B

```
@RestController
public class DiseaseController {

    private final DiseaseService diseaseService;

    @Autowired
    public DiseaseController(DiseaseService diseaseService){
        this.diseaseService = diseaseService;
    }

    @GetMapping("/api/v1/diseases")
    @ResponseStatus(HttpStatus.OK)
    public ResponseEntity<List<Disease>> getAllDiseases(){
        return new
ResponseEntity<>(diseaseService.findDiseases(),
HttpStatus.OK);
    }

    @GetMapping("api/v1/diseases/{diseaseId}")
    @ResponseStatus(HttpStatus.OK)
    public ResponseEntity<Disease> getDisease(@PathVariable
Integer diseaseId){
        Disease disease =
diseaseService.findDiseaseById(diseaseId);
        if (disease == null) throw new
ResourceNotFoundException("disease not found");
        return new ResponseEntity<>(disease, HttpStatus.OK);
    }

}
```

SecurityConfiguration.java

```
.requestMatchers(HttpMethod.GET,
"/api/v1/diseases/**").hasAuthority("VET")
```

TEST 4

```
@Getter
@Setter
@Entity
public class Symptom extends BaseEntity {

    @NotNull
    @NotBlank
    @Size(min = 3, max = 50)
    String name;

    @Pattern(regexp = "^(LOW)|(MEDIUM)|(HIGH)$")
    String virulence;

    @ManyToMany
    @NotNull
    Set<Disease> includes;

}
```

```
@Getter
@Setter
@Entity
public class Treatment extends BaseEntity {
    @NotNull
    @NotBlank
    @Size(min = 3, max = 50)
    String name;

    String description;

    @NotNull
    @Min(1)
    Integer baseDose;

    @Min(1)
    Integer shockDose;

    @NotNull
    @Min(1)
    Integer maxDose;
```

```
    @ManyToMany
    @NotNull
    Set<Disease> recommendedFor;
}
```

```
public interface SymptomRepository extends
CrudRepository<Symptom, Integer> {

    Optional<Symptom> findById(Integer i);

    List<Symptom> findAll();

    Symptom save(Symptom any);

}
```

```
public interface TreatmentRepository extends
CrudRepository<Treatment, Integer> {

    Optional<Treatment> findById(Integer i);

    List<Treatment> findAll();

    Treatment save(Treatment any);

}
```

TEST 5

```
INSERT INTO symptom(id, name, virulence) VALUES (1, 'Cough', 'LOW'), (2,
'Hair loss', 'MEDIUM');
INSERT INTO treatment(id, name, base_dose, shock_dose, max_dose,
description) VALUES
(1, 'aspirin', 12, null, 50, null), (2, 'paracetamol', 20, 40, 150, null);
INSERT INTO  treatment_recommended_for( treatment_id, recommended_for_id)
VALUES (1, 2), (2, 1);
INSERT INTO symptom_includes(symptom_id, includes_id) VALUES (1, 2), (1,
3), (2, 1), (2, 3);
INSERT INTO visits_symptoms(visit_id, symptoms_id) VALUES (1, 1), (1, 2);
```